

PERCOBAAN 4

STATEMENT PENENTU KEPUTUSAN

4.1 Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan konsep statement penentu keputusan dalam program.
- Menggunakan statement **if** untuk menjalankan proses berdasarkan suatu kondisi.
- Menggunakan **if-else** untuk memilih salah satu dari dua kemungkinan proses.
- Menggunakan **else-if** untuk memilih satu kondisi dari beberapa kondisi yang tersedia.
- Menggunakan compound statement untuk mengelompokkan beberapa statement dalam satu blok.
- Menggunakan **switch-case** sebagai alternatif pemilihan berdasarkan nilai tertentu.
- Menggunakan operator relasional dan operator logika dalam pembentukan kondisi.
- Menguji program dengan beberapa nilai masukan dan menjelaskan hasil eksekusinya.

4.2 Dasar Teori

Statement penentu keputusan digunakan untuk mengatur alur eksekusi program berdasarkan kondisi tertentu. Kondisi dievaluasi sebagai benar (true) atau salah (false). Jika kondisi terpenuhi, program menjalankan statement yang sesuai; jika tidak terpenuhi, program dapat menjalankan statement alternatif.

Kondisi pada statement penentu keputusan umumnya dibentuk menggunakan operator relasional dan/atau operator logika. Beberapa bentuk yang digunakan pada praktikum ini adalah if, if-else, else-if, compound statement, nested if, dan switch-case.

4.2.1 Operator Relasional

Operator	Makna	Contoh
==	sama dengan	a == b
!=	tidak sama dengan	a != b
>	lebih besar	a > b
<	lebih kecil	a < b
>=	lebih besar atau sama dengan	a >= b
<=	lebih kecil atau sama dengan	a <= b

4.2.2 Operator Logika

Operator	Makna	Contoh
&&	AND; semua kondisi harus benar	a > 0 && a < 10
 	OR; minimal satu kondisi benar	a < 0 a > 100
!	NOT; membalik nilai logika	! (a == 0)

4.2.3 Statement if

Statement if digunakan ketika suatu proses hanya perlu dijalankan apabila kondisi tertentu terpenuhi.

```
if (kondisi) {  
    statement;  
}
```

4.2.4 Statement if-else

Statement if-else digunakan ketika terdapat dua kemungkinan proses: satu proses dijalankan jika kondisi benar dan proses lainnya jika kondisi salah.

```
if (kondisi) {  
    statement_jika_benar;  
} else {  
    statement_jika_salah;  
}
```

4.2.5 Statement else-if

Statement else-if digunakan ketika terdapat beberapa kondisi yang perlu diperiksa secara berurutan. Program menjalankan blok pertama yang kondisinya terpenuhi.

```
if (kondisi_1) {  
    statement_1;  
} else if (kondisi_2) {  
    statement_2;  
} else {  
    statement_lain;  
}
```

4.2.6 Nested if

Nested if adalah penggunaan statement if di dalam statement if lainnya. Bentuk ini dapat digunakan ketika keputusan kedua baru relevan setelah keputusan pertama terpenuhi.

```
if (kondisi_1) {  
    if (kondisi_2) {  
        statement;  
    }  
}
```

4.2.7 Compound Statement

Compound statement adalah beberapa statement yang dikelompokkan dalam satu blok menggunakan tanda kurung kurawal { }. Pengelompokan ini penting agar beberapa proses diperlakukan sebagai satu bagian dari if, else, atau struktur keputusan lainnya.

4.2.8 Statement switch-case

Statement switch-case digunakan untuk memilih proses berdasarkan nilai suatu ekspresi. Setiap nilai pilihan ditulis dalam case. Statement break digunakan untuk menghentikan eksekusi pada case yang sesuai, sedangkan default dijalankan apabila tidak ada case yang cocok.

Bagian	Fungsi
switch	Menentukan ekspresi yang diperiksa.
case	Menentukan nilai pilihan.
break	Menghentikan eksekusi switch.
default	Menangani kondisi ketika tidak ada case yang sesuai.
Catatan: Pada modul ini digunakan sintaks C yang lebih umum, yaitu <code>int main()</code> , <code>#include <stdio.h></code> , dan <code>return 0</code> . Fungsi <code>getch()</code> dari <code>conio.h</code> tidak digunakan karena bukan bagian dari pustaka standar C.	

4.3 Alat dan Bahan

- Komputer/laptop.
- Compiler atau IDE yang mendukung bahasa C.
- Editor kode sumber.
- Modul praktikum.

4.4 Program Percobaan

4.4.1 Percobaan 1 – Statement if

Program berikut menentukan nilai absolut suatu bilangan. Jika bilangan bernilai negatif, tandanya diubah menjadi positif. Jika tidak negatif, nilainya tetap.

```
#include <stdio.h>

int main(void)
{
    int bil, absolut;

    printf("Masukkan bilangan bulat: ");
    scanf("%d", &bil);

    absolut = bil;
    if (bil < 0)
        absolut = -bil;

    printf("Nilai absolut dari %d adalah %d\n", bil, absolut);

    return 0;
}
```

Analisis program:

- **Input:** Satu bilangan bulat.
- **Kondisi:** `bil < 0`.

- **Proses:** Jika kondisi benar, absolut diisi -bil.
- **Output:** Nilai absolut bilangan.
- **Konsep:** Statement if.

4.4.2 Percobaan 2 – if-else dan Operator Modulus

Operator modulus (%) menghasilkan sisa pembagian bilangan bulat. Program menggunakan sisa pembagian untuk menentukan apakah pembagian habis atau tidak.

```
#include <stdio.h>

int main(void)
{
    int pembilang, penyebut, sisa;

    printf("Masukkan pembilang: ");
    scanf("%d", &pembilang);

    printf("Masukkan penyebut: ");
    scanf("%d", &penyebut);

    if (penyebut == 0) {
        printf("Penyebut tidak boleh 0.\n");
    } else {
        sisa = pembilang % penyebut;

        if (sisa != 0)
            printf("%d tidak habis dibagi %d.\n", pembilang, penyebut);
        else
            printf("%d habis dibagi %d.\n", pembilang, penyebut);
    }

    return 0;
}
```

Analisis program:

- **Input:** Pembilang dan penyebut.
- **Kondisi 1:** Penyebut == 0 untuk mencegah pembagian dengan nol.
- **Kondisi 2:** sisa != 0 untuk menentukan pembagian tidak habis.
- **Output:** Keterangan hasil pembagian.
- **Konsep:** if-else, nested if, dan operator modulus.

4.4.3 Percobaan 3 – Kondisi pada Karakter

Program berikut memeriksa apakah karakter yang dimasukkan termasuk huruf kecil a sampai z. Pemeriksaan dilakukan dengan operator relasional dan logika AND (&&).

```
#include <stdio.h>

int main(void)
```

```

{
    char c;

    printf("Masukkan suatu karakter: ");
    scanf(" %c", &c);

    if (c >= 'a' && c <= 'z')
        printf("Karakter tersebut termasuk huruf kecil.\n");
    else
        printf("Karakter tersebut bukan huruf kecil.\n");

    return 0;
}

```

Analisis program:

- **Input:** Satu karakter.
- **Kondisi:** `c >= 'a' && c <= 'z'`.
- **Proses:** Kedua kondisi harus benar agar karakter dianggap huruf kecil.
- **Output:** Keterangan kategori karakter.
- **Konsep:** if-else dan operator logika.

4.4.4 Percobaan 4 – Demonstrasi goto

Percobaan ini mempertahankan konsep pada modul awal, yaitu kombinasi if dan goto untuk mengulang proses. Contoh menghitung kelipatan 2 secara berulang sampai iterasi ke-10.

```

#include <stdio.h>

int main(void)
{
    int i = 0, hasil = 1;

ulang:
    i++;
    hasil *= 2;

    printf("Iterasi ke-%2d: %5d\n", i, hasil);

    if (i < 10)
        goto ulang;

    return 0;
}

```

Analisis program:

- **Input:** Tidak ada input dari pengguna.
- **Proses:** Nilai hasil dikalikan 2 setiap kali label ulang dicapai.

- **Kondisi:** $i < 10$.
- **Output:** Nilai hasil pada setiap iterasi.
- **Konsep:** if dan goto.

Catatan: goto ditampilkan karena merupakan bagian dari materi sumber. Dalam pengembangan program, struktur perulangan for, while, atau do-while umumnya lebih mudah dibaca dan dipelihara untuk kasus perulangan seperti ini.

4.4.5 Percobaan 5 – Compound Statement

Compound statement digunakan ketika satu kondisi harus menjalankan beberapa statement sekaligus. Contoh berikut menentukan pilihan merek berdasarkan tipe kendaraan.

```
#include <stdio.h>

int main(void)
{
    char tag;

    printf("Masukkan tipe tag (2 / 4): ");
    scanf(" %c", &tag);

    if (tag == '2') {
        printf("Merek sepeda motor bertipe 2 tag:\n");
        printf("1. YAMAHA\n");
        printf("2. SUZUKI\n");
        printf("3. HONDA\n");
    } else {
        printf("Merek sepeda motor bertipe 4 tag:\n");
        printf("1. HONDA\n");
        printf("2. SUZUKI\n");
        printf("3. YAMAHA\n");
    }

    return 0;
}
```

Analisis program:

- **Input:** Karakter tipe tag.
- **Kondisi:** $tag == '2'$.
- **Proses:** Setiap cabang menjalankan beberapa printf dalam satu blok.
- **Output:** Daftar merek sesuai cabang kondisi.
- **Konsep:** if-else dan compound statement.

4.4.6 Percobaan 6 – else-if

Program berikut memilih operasi aritmatika berdasarkan operator yang dimasukkan pengguna. Struktur else-if digunakan untuk memeriksa beberapa kemungkinan secara berurutan.

```

#include <stdio.h>

int main(void)
{
    char operator;
    float bil1, bil2, hasil;

    printf("Masukkan: bilangan1 operator bilangan2\n");
    printf("Operator: + - * /\n");
    printf("Input: ");

    scanf("%f %c %f", &bil1, &operator, &bil2);

    if (operator == '*')
        hasil = bil1 * bil2;
    else if (operator == '/')
        hasil = bil1 / bil2;
    else if (operator == '+')
        hasil = bil1 + bil2;
    else if (operator == '-')
        hasil = bil1 - bil2;
    else {
        printf("Operator tidak dikenali.\n");
        return 0;
    }

    printf("Hasil: %.2f %c %.2f = %.2f\n",
        bil1, operator, bil2, hasil);

    return 0;
}

```

Analisis program:

- **Input:** Dua bilangan dan satu operator.
- **Kondisi:** Operator dibandingkan dengan *, /, +, dan -.
- **Proses:** Satu operasi yang sesuai dijalankan.
- **Output:** Hasil operasi aritmatika atau pesan kesalahan.
- **Konsep:** else-if dan compound statement.

Catatan: Untuk penyempurnaan program, tambahkan pemeriksaan $bil2 \neq 0$ sebelum melakukan pembagian.

4.4.7 Percobaan 7 – switch-case

Program berikut merupakan alternatif dari percobaan sebelumnya. Pemilihan operasi dilakukan menggunakan switch-case.

```

#include <stdio.h>

```

```

int main(void)
{
    char operator;
    float bil1, bil2, hasil;

    printf("Masukkan: bilangan1 operator bilangan2\n");
    printf("Operator: + - * /\n");
    printf("Input: ");

    scanf("%f %c %f", &bil1, &operator, &bil2);

    switch (operator) {
        case '*':
            hasil = bil1 * bil2;
            break;
        case '/':
            if (bil2 == 0) {
                printf("Pembagian dengan nol tidak diperbolehkan.\n");
                return 0;
            }
            hasil = bil1 / bil2;
            break;
        case '+':
            hasil = bil1 + bil2;
            break;
        case '-':
            hasil = bil1 - bil2;
            break;
        default:
            printf("Operator tidak dikenali.\n");
            return 0;
    }

    printf("Hasil: %.2f %c %.2f = %.2f\n",
        bil1, operator, bil2, hasil);

    return 0;
}

```

Analisis program:

- **Input:** Dua bilangan dan satu operator.
- **Proses:** switch memilih case berdasarkan nilai operator.
- **break:** Menghentikan eksekusi setelah case yang sesuai.
- **default:** Menangani operator yang tidak tersedia.
- **Konsep:** switch-case, break, default, dan compound statement.

4.5 Pengujian Program

Setiap program harus diuji dengan beberapa data masukan. Pengujian tidak hanya menggunakan data normal, tetapi juga kondisi batas dan data yang berpotensi menghasilkan kesalahan.

No.	Data Uji	Hasil yang Diharapkan	Hasil Aktual
1			
2			
3			
4			
5			

4.6 Analisis Hasil Praktikum

Jawablah pertanyaan berikut berdasarkan program yang telah dijalankan.

1. Apa perbedaan utama antara if dan if-else?

Jawaban: _____

2. Kapan struktur else-if lebih sesuai digunakan daripada beberapa if yang berdiri sendiri?

Jawaban: _____

3. Mengapa operator && pada percobaan karakter menyebabkan kedua kondisi harus terpenuhi?

Jawaban: _____

4. Apa fungsi tanda kurung kurawal { } pada compound statement?

Jawaban: _____

5. Apa fungsi break pada switch-case?

Jawaban: _____

6. Apa yang terjadi jika break dihilangkan dari sebuah case?

Jawaban: _____

7. Mengapa pembagian dengan nol perlu ditangani secara khusus?

Jawaban: _____

8. Apa kelebihan dan kekurangan penggunaan goto pada contoh percobaan?

Jawaban: _____

4.7 Tugas Praktikum

Tugas dikerjakan secara mandiri. Setiap program harus disertai kode sumber, minimal tiga data uji, hasil eksekusi, dan penjelasan singkat mengenai logika percabangannya.

4.7.1 Bilangan Ganjil atau Genap

Buat program untuk menentukan apakah bilangan bulat yang dimasukkan melalui keyboard merupakan bilangan ganjil atau genap.

4.7.2 Klasifikasi Karakter

Buat program untuk menentukan kelompok karakter: huruf kecil, huruf besar, angka, atau karakter khusus.

4.7.3 Kalkulator Sederhana

Buat program kalkulator sederhana dengan dua bilangan dan satu operator +, -, *, atau /. Program harus menangani operator yang tidak valid dan pembagian dengan nol.

4.7.4 Tahun Kabisat

Buat program untuk menentukan apakah suatu tahun merupakan tahun kabisat. Program menerima tahun dari keyboard dan menampilkan hasil keputusan.

4.7.5 Kategori Nilai

Buat program yang menerima nilai 0–100 dan menentukan kategori: A (85–100), B (75–84), C (65–74), D (50–64), dan E (<50). Program harus menolak nilai di luar rentang 0–100.

4.7.6 Akar Persamaan Kuadrat

Buat program untuk menghitung determinan $D = B^2 - 4AC$ dan menentukan jenis akar persamaan $Ax^2 + Bx + C = 0$ berdasarkan nilai D . Tangani juga kondisi $A = 0$.

4.7.6 Ketentuan Tugas Akar Persamaan Kuadrat

- Jika $D > 0$, terdapat dua akar real berbeda.
- Jika $D = 0$, terdapat satu akar real kembar.
- Jika $D < 0$, terdapat dua akar kompleks.
- Jika $A = 0$, persamaan bukan persamaan kuadrat dan program harus memberikan pesan yang sesuai.

Gunakan rumus:

$$D = B^2 - 4AC$$

$$X_{1,2} = (-B \pm \sqrt{D}) / (2A) \text{ untuk } D \geq 0$$

4.8 Format Laporan Praktikum

Laporan praktikum disusun dengan urutan berikut:

- Halaman judul.
- Tujuan praktikum.
- Dasar teori singkat.
- Kode program setiap percobaan.
- Hasil running program/screenshot.
- Tabel pengujian dan hasil aktual.

- Analisis hasil percobaan.
- Jawaban pertanyaan analisis.
- Kode dan hasil tugas praktikum.
- Kesimpulan.

Pada bagian analisis, mahasiswa tidak cukup hanya menuliskan hasil running. Jelaskan hubungan antara input, kondisi, proses percabangan, dan output yang dihasilkan.

4.9 Kesimpulan

Statement penentu keputusan memungkinkan program memilih alur proses berdasarkan kondisi yang diberikan. Melalui praktikum ini, mahasiswa mempelajari penggunaan if, if-else, else-if, nested if, compound statement, goto, dan switch-case. Pemilihan struktur keputusan harus disesuaikan dengan bentuk masalah agar program mudah dipahami dan diuji.

4.10 Checklist Praktikum

- ☐ Semua program berhasil dikompilasi.
- ☐ Setiap program telah diuji dengan beberapa data.
- ☐ Hasil running telah didokumentasikan.
- ☐ Analisis setiap percobaan telah dikerjakan.
- ☐ Semua tugas telah dikerjakan.
- ☐ Kode program telah diberi indentasi dan komentar seperlunya.